

Git 기초 및 활용 마스터 과정

비전공 재직자를 위한 실무 중심 Git 명령어와 C++ 예제 완벽 가이드

DevOps Education

인포그래픽: 분산 버전 관리 시스템

- ▶ **안전한 백업과 분산화:** 중앙 서버가 손상되더라도 각 개발자의 로컬 컴퓨터에 전체 프로젝트의 완벽한 복사본이 존재합니다.
- ▶ **독립적인 작업 환경:** 네트워크에 연결되어 있지 않아도 로컬 환경에서 자유롭게 버전을 관리하고 커밋할 수 있습니다.
- ▶ **효율적인 팀 협업:** 서로의 작업물에 영향을 주지 않으면서 코드를 수정하고, 필요할 때 클라우드를 통해 안전하게 병합합니다.

명령어 설명: 초기 환경 설정

사용자 정보 등록

- ▶ Git은 모든 커밋에 작성자의 이름과 이메일을 기록합니다.

```
$ git config --global user.name "Your Name"
```

```
$ git config --global user.email "you@example.com"
```

설정 내역 확인

- ▶ 입력한 정보가 시스템에 정상적으로 저장되었는지 검증합니다.

```
$ git config --list
```

실습 예제: Git 환경 준비

환경 준비 및 확인

```
$ git config --global user.name "Dev Kim"  
$ git config --global user.email "dev.kim@company.com"  
$ git config --list
```

터미널 출력 예시:

```
credential.helper=manager  
core.autocrlf=true  
...  
user.name=Dev Kim  
user.email=dev.kim@company.com
```

연습 문제: 나만의 Git 환경 세팅하기

- ▶ **터미널 실행:** 컴퓨터에서 터미널 애플리케이션을 실행하세요.
- ▶ **이름 설정:** `git config --global user.name` 명령어 뒤에 본인의 영문 이름이나 닉네임을 따옴표 안에 넣어 실행하세요.
- ▶ **이메일 설정:** `git config --global user.email` 명령어 뒤에 주로 사용하는 이메일 주소를 넣어 실행하세요.
- ▶ **검증:** `git config --list` 명령어를 입력하여 설정한 두 항목이 리스트 하단에 제대로 위치해 있는지 확인하세요.

인포그래픽: Git의 3가지 상태 영역

- ▶ **Working Directory (작업 디렉토리):** 실제 코드를 작성하고 수정하는 눈에 보이는 폴더 영역입니다.
- ▶ **Staging Area (스테이징 영역):** 다음 버전에 포함시킬 수정된 파일들만 선별해서 올려두는 가상의 준비 공간입니다.
- ▶ **Repository (로컬 저장소):** 스테이징된 파일들이 영구적인 스냅샷(커밋)으로 기록되어 보관되는 안전한 금고입니다.

명령어 설명: 저장소 초기화 및 상태 확인

저장소 생성 (Init)

- ▶ 일반 폴더를 Git이 버전 관리를 할 수 있는 저장소로 변환합니다.

```
$ git init
```

상태 확인 (Status)

- ▶ 프로젝트 폴더 내 파일들의 현재 상태를 가장 직관적으로 보여줍니다.

```
$ git status
```

명령어 설명: 스테이징과 커밋

장바구니 담기 (Add)

```
$ git add <file>  
$ git add .
```

- ▶ `add .`은 현재 디렉토리의 수정되거나 추가된 모든 파일을 한 번에 스테이징합니다.

영구 저장 (Commit)

```
$ git commit -m "메시지"
```

- ▶ `-m` 옵션은 에디터를 열지 않고 즉시 짧은 설명을 첨부합니다.

실습 예제: C++ 첫 코드 커밋하기

C++ 프로젝트 시작

```
$ mkdir my_cpp_project  
$ cd my_cpp_project  
$ git init
```

main.cpp 파일 작성 및 커밋

```
#include <iostream>  
int main() {  
    std::cout << "Git Starter" << std::endl;  
    return 0;  
}
```

```
$ git add main.cpp  
$ git commit -m "초기 설정: main 함수 뼈대 작성"
```

연습 문제: 계산기 뼈대 만들기

- ▶ **프로젝트 초기화:** 빈 폴더를 하나 만들고, 터미널에서 진입한 뒤 `git init` 명령어를 실행하세요.
- ▶ **코드 파일 작성:** `calculator.cpp` 파일을 만들고 덧셈을 수행하는 간단한 함수를 작성하여 저장하세요.
- ▶ **추적 상태 확인:** `git status`를 입력해 파일이 아직 추적되지 않고 있음을 확인하세요.
- ▶ **스테이징 영역 추가:** `git add .` 명령어를 사용하여 변경 사항을 추가하세요.
- ▶ **커밋 생성:** `git commit -m "feat: 덧셈 기능 모듈 추가"` 라고 입력하여 프로젝트의 첫 번째 스냅샷을 완성하세요.

인포그래픽: 작업 이력과 변경 추적

- ▶ **타임라인의 형성:** 커밋을 반복할 때마다 이전 스냅샷과 연결된 하나의 거대한 타임라인(히스토리)이 만들어집니다.
- ▶ **고유한 해시(Hash):** 모든 커밋은 절대 중복되지 않는 40자리의 알파벳+숫자 조합(SHA-1 해시)을 부여받아 완벽하게 추적됩니다.
- ▶ **비교와 복구:** 현재 작업 중인 코드와 과거의 코드가 정확히 어떻게 다른지 비교할 수 있으며, 원한다면 과거로 타임머신을 탈 수 있습니다.

명령어 설명: 로그 확인 및 비교

과거 기록 조회 (Log)

```
$ git log
```

```
$ git log --oneline
```

- ▶ 작성자, 날짜, 커밋 해시값, 메시지를 상세히 혹은 한 줄로 요약하여 출력합니다.

변경 사항 상세 비교 (Diff)

```
$ git diff
```

- ▶ 아직 스테이징하지 않은 코드와 마지막 커밋 코드 간의 텍스트 차이를 줄 단위로 표시합니다.

명령어 설명: 변경 사항 취소 및 되돌리기

작업 취소하기 (Restore)

```
$ git restore <file>
```

- ▶ 아직 커밋하지 않은 파일의 수정을 취소하고 원래대로 복구합니다.

과거 시점으로 이동 (Checkout)

```
$ git checkout <commit_hash>
```

- ▶ 특정 커밋 번호로 전체 프로젝트의 시간을 되돌립니다. 예전 코드를 열어보고 분석할 때 유용합니다.

실습 예제: C++ 코드 비교 및 복구

의도적 버그 삽입 및 비교

```
int main() {  
    // std::cout << "Git Starter" << std::endl;  
    int a = 10 / 0; // 치명적 오류 추가  
    return 0;  
}
```

```
$ git diff
```

실수 되돌리기

```
$ git restore main.cpp
```

- ▶ 에디터 확인 시 치명적인 오류 코드가 사라지고 마지막 버전으로 돌아옵니다.

연습 문제: 타임머신 탑승하기

- ▶ **히스토리 조회:** `git log --oneline`을 실행하여 짧은 해시값을 확인하세요.
- ▶ **코드 망치기:** `calculator.cpp` 파일의 덧셈 기호를 곱셈으로 바꾼 뒤 파일을 저장하세요.
- ▶ **차이점 분석:** `git diff`를 입력하여 변경된 줄을 확인하세요.
- ▶ **즉시 복원:** `git restore calculator.cpp` 명령어를 실행하여 코드를 마지막 커밋 상태로 복구하세요.
- ▶ **과거 탐험:** `git checkout [해시값]`을 입력하여 초기 상태로 돌아간 후, 다시 `git checkout main`으로 복귀하세요.

인포그래픽: 브랜치(Branch)의 개념

- ▶ **병렬 우주 생성:** 현재의 안정적인 코드를 훼손하지 않고, 동일한 코드 베이스에서 가지를 쳐서 안전한 격리 공간을 만듭니다.
- ▶ **실험과 버그 픽스:** 새로운 C++ 라이브러리를 도입하거나 로직을 갈아엎는 위험한 작업도 브랜치 안에서는 마음껏 테스트할 수 있습니다.
- ▶ **자유로운 이동:** 명령어 한 줄이면 A 기능 개발 환경에서 B 버그 수정 환경으로 순식간에 작업 폴더의 파일 구성을 교체할 수 있습니다.

명령어 설명: 브랜치 생성 및 이동

브랜치 목록 및 생성 (Branch)

```
$ git branch
```

```
$ git branch <name>
```

- ▶ 브랜치 목록을 보여주거나 새로운 이름의 가지를 생성합니다.

브랜치 전환 (Switch / Checkout)

```
$ git switch <name>
```

```
$ git switch -c <name>
```

- ▶ -c 옵션을 붙이면 새로운 브랜치를 생성함과 동시에 해당 브랜치로 즉시 이동합니다.

실습 예제: 브랜치 분리하여 C++ 개발하기

기능 브랜치 생성 및 작업

```
$ git switch -c feature/multiply
```

```
// calculator.cpp 끝에 함수 추가
```

```
int multiply(int a, int b) {  
    return a * b;  
}
```

```
$ git add calculator.cpp
```

```
$ git commit -m "feat: 곱셈 기능 추가"
```

브랜치 전환 확인

```
$ git switch main
```

연습 문제: 나만의 실험실 만들기

- ▶ **브랜치 확인:** `git branch`를 입력하여 현재 브랜치가 `main` 하나뿐인지 확인하세요.
- ▶ **생성과 이동:** `git switch -c feature/divide` 명령어로 독립 공간을 만드세요.
- ▶ **기능 구현:** 에디터에서 두 수를 나누는 C++ 함수를 작성하고 파일을 저장하세요.
- ▶ **커밋으로 확정:** 수정한 내용을 `add` 하고 메시지와 함께 커밋을 완료하세요.
- ▶ **차이점 체감:** `git switch main`으로 돌아가서 코드가 이전 상태로 복구되었는지 눈으로 확인하세요.

인포그래픽: 병합(Merge)과 충돌(Conflict)

- ▶ **병합의 목적:** 브랜치에서 안전하게 테스트를 마친 완성된 기능이나 버그 픽스를 다시 원본 프로젝트에 합칩니다.
- ▶ **충돌 발생 원인:** main 브랜치와 feature 브랜치에서 동일한 파일의 정확히 같은 줄을 서로 다르게 수정했을 때 발생합니다.
- ▶ **수동 해결:** 충돌이 난 파일에는 특수한 마커(<<<<, ==, >>>>)가 생기며, 개발자가 직접 원하는 코드를 남기고 해결합니다.

명령어 설명: 브랜치 병합하기

다른 가지 끌어오기 (Merge)

```
$ git checkout main
```

```
$ git merge <target_branch>
```

- ▶ **규칙:** 합쳐질 내용이 들어올 "받을 브랜치"로 먼저 이동한 후에 명령어를 실행합니다.

충돌 마커 읽는 법

```
<<<<<< HEAD (현재 main)
```

```
int result = a + b + 10;
```

```
=====
```

```
int result = a + b * 2;
```

```
>>>>>> feature/calc (가져온 브랜치)
```

실습 예제: C++ 기능 브랜치 병합

정상적인 병합 (Fast-forward)

```
$ git switch main  
$ git merge feature/multiply  
$ git branch -d feature/multiply
```

충돌이 발생했을 경우 해결 과정

```
$ git merge feature/conflict_test  
// 에디터에서 마커를 지우고 C++ 코드 정리 후  
$ git add .  
$ git commit -m "fix: 병합 충돌 해결"
```

연습 문제: 고의로 충돌 내고 수습하기

- ▶ **main 조작:** main 브랜치 calculator.cpp 첫 줄에 주석 // 버전 1.0을 추가하고 커밋합니다.
- ▶ **브랜치 조작:** feature/divide 브랜치로 이동하여 첫 줄에 // 나눗셈 버전이라고 주석을 달고 커밋합니다.
- ▶ **충돌 유발:** main 브랜치로 돌아가서 merge를 실행하여 에러를 발생시키세요.
- ▶ **코드 병합:** 파일을 열어 새로운 주석으로 코드를 직접 수정하고 마커를 삭제하세요.
- ▶ **해결 선언:** 수정된 파일을 add 하고 commit 하여 마무리 지으세요.

인포그래픽: 클라우드로 나아가기 (원격 저장소)

- ▶ **GitHub의 역할:** 내 컴퓨터에만 있던 Git 저장소를 클라우드 서버(Remote)에 올려 전 세계 어디서든 접근 가능하게 만듭니다.
- ▶ **안전한 백업:** 랜섬웨어에 걸리거나 장비를 분실해도, GitHub에 Push해둔 코드가 있다면 즉시 원상태로 복원할 수 있습니다.
- ▶ **팀 워크의 핵심:** 수십 명의 개발자가 각자의 컴퓨터에서 작업한 내용을 중앙 원격 서버를 통해 하나로 합치고 코드를 리뷰합니다.

명령어 설명: 원격 연결 및 복제

원격 저장소 주소 연결 (Remote)

```
$ git remote add origin <URL>
```

```
$ git remote -v
```

- ▶ GitHub에서 생성한 리포지토리의 주소를 origin이라는 이름으로 로컬과 연결합니다.

남의 프로젝트 통째로 받기 (Clone)

```
$ git clone <URL>
```

- ▶ 클라우드에 있는 프로젝트 소스코드와 커밋 히스토리를 내 컴퓨터로 한 번에 복제합니다.

명령어 설명: 클라우드 동기화

클라우드에 업로드 (Push)

```
$ git push origin main
```

```
$ git push -u origin main
```

- ▶ 로컬에서 쌓아둔 커밋들을 원격 서버로 전송합니다. -u 옵션은 향후 기본 연결을 세팅합니다.

최신 변경사항 내려받기 (Pull)

```
$ git pull origin main
```

- ▶ 다른 팀원이 올려둔 코드를 내 컴퓨터로 갱신하고 현재 브랜치에 자동으로 병합합니다.

실습 예제: C++ 프로젝트 GitHub에 올리기

GitHub 리포지토리 생성 및 연결

- ▶ GitHub 웹사이트에서 'New repository' 생성 후 주소 복사

```
$ git remote add origin https://github.com/myID/cpp_study.git
```

서버로 코드 전송

```
$ git push -u origin main
```

- ▶ 명령어 실행 후 GitHub 페이지를 새로고침하면 웹상에 업로드된 C++ 파일들을 확인할 수 있습니다.

연습 문제: Pull Request 참여하기

- ▶ **프로젝트 복제:** `git clone [URL]`을 실행해 팀원의 코드를 내려받으세요.
- ▶ **새 가지치기:** `git switch -c feature/my-update`로 브랜치를 만드세요.
- ▶ **수정 및 커밋:** C++ 파일 내에 닉네임을 출력하는 코드를 추가하고 커밋하세요.
- ▶ **푸시하기:** `git push origin feature/my-update`를 실행해 업로드하세요.
- ▶ **PR 요청:** GitHub 웹 페이지에서 **Compare & pull request** 버튼을 눌러 코드 리뷰를 요청하세요.

인포그래픽: 복잡한 상황 해결 도구

- ▶ **Git Stash (보관함):** 코딩 중 문법 에러가 나는 상태인데 치명적인 버그를 수정해 달라는 요청이 들어올 때, 더러운 코드를 '임시 보관함'에 밀어 넣고 작업 환경을 청소합니다.
- ▶ **Git Rebase (히스토리 재배치):** 거미줄처럼 복잡하게 꼬인 여러 브랜치의 커밋 내역들을 깔끔하게 일직선으로 정리합니다. Merge보다 깔끔한 로그를 원할 때 사용합니다.

명령어 설명: Stash & Rebase / Q&A

Stash (작업 치워두기 및 복원)

```
$ git stash
```

```
$ git stash pop
```

- ▶ 에러가 나는 코드를 임시 저장하고 마지막 커밋으로 돌아갑니다. pop으로 다시 복원합니다.

Rebase (히스토리 정리)

```
$ git rebase main
```

- ▶ 현재 브랜치의 베이스를 main의 최신 커밋 위로 옮깁니다.

수고하셨습니다! Q&A (질문이나 실습 중 막히는 부분을 알려주세요)